

Network Cache Lookup Optimization using Bloom Filters

Fianl-Project Report

Murad Imre - mil5

Abstract

Modern web infrastructure, such as routers, content delivery networks (CDNs), and distributed caching systems, relies on fast lookup operations to handle billions of requests per second. However, deterministic cache structures often incur high memory overhead and latency when managing large and dynamic datasets. This project explores how probabilistic data structures, specifically Bloom filters and their variants, can improve cache lookup efficiency by serving as lightweight pre-filters that quickly reject non-existent entries before costly cache or database access.

We implemented and benchmarked three algorithms: the classical Bloom Filter, the Counting Bloom Filter, and the Cuckoo Filter. Using Zipf-distributed request workloads to model realistic web and DNS traffic, we evaluated each filter across different bit array sizes (m), numbers of hash functions (k), and load factors. Metrics such as false positive rate (FPR), memory usage (bits per key), average query latency, backend lookups per query, and end-to-end throughput were measured to study trade-offs between accuracy, speed, and memory efficiency.

The goal is to identify conditions under which probabilistic pre-filters provide the best balance between accuracy, speed, and memory efficiency in network caching. We also compare these probabilistic filters against a deterministic baseline (a hash-based cache index without a pre-filter) to quantify throughput gains and backend lookup reductions under realistic Zipf workloads.

Problem Description

Modern distributed caching systems, such as CDNs, DNS resolvers, and web proxies, continuously perform membership lookups to determine whether a requested object is stored in cache before accessing slower storage tiers. At scale, these systems handle tens of millions of queries per second, making lookup efficiency a critical performance factor. Deterministic structures like hash tables offer exact membership tests but become memory-intensive as datasets grow, and when tables exceed CPU cache, randomized accesses lead to degraded locality, higher latency, and increased cache misses.

Probabilistic data structures such as Bloom Filters, Counting Bloom Filters, and Cuckoo Filters seek to mitigate these issues by providing small, fixed-memory “pre-filters” that quickly rule out absent keys. However, these structures introduce meaningful trade-offs: Bloom filters do not support deletions and suffer higher false positive rates at high load; Counting Bloom Filters allow deletions but increase memory usage via counters; and Cuckoo Filters offer deletion and improved space efficiency but may fail insertions at high load factors. Since real-world traffic follows Zipfian distributions, where a small set of keys dominates requests, the interaction between workload skew, filter parameters, and memory constraints becomes complex. A systematic comparative evaluation

is necessary to understand which probabilistic structure achieves the best balance across accuracy, latency, and memory usage in realistic caching scenarios.

Literature Survey

Efficient membership testing has long been recognized as a performance bottleneck in large-scale storage systems, including Google Bigtable [1], Amazon Dynamo [2], and Cassandra [3]. These systems highlight memory pressure and lookup latency as central challenges. Even with distributed balancing approaches such as consistent hashing [4], the cost of per-node membership testing remains significant. Course discussions have likewise emphasized that storing exact sets, as in malicious URL detection, is often infeasible at web scale, motivating memory-efficient probabilistic alternatives.

Bloom Filters, introduced by Bloom in 1970 [5], provide a compact method for membership testing that guarantees no false negatives. They have become widely used in practice, appearing in storage engines such as Bigtable, HBase, and Cassandra. Broder and Mitzenmacher’s survey [6] formalizes their role in networking, caching, and distributed systems. However, classical Bloom filters do not support deletions and degrade when overloaded.

Counting Bloom Filters, introduced by Fan et al. in early web caching work [7], replace bits with small counters to support deletions, trading space for flexibility. More recently, Cuckoo Filters [8] leverage cuckoo hashing and compact fingerprints to enable deletions while achieving lower false positive rates at similar or lower memory cost. Empirical and theoretical comparisons (e.g., Fan 2014, Mitzenmacher 2018) show that the best filter depends on update frequency, workload skew, and memory budget. Since real-world access patterns often follow Zipf distributions, a controlled comparison under such workloads is critical to understanding practical performance.

Hypothesis

We hypothesize that Cuckoo Filters will outperform Bloom and Counting Bloom Filters in lookup latency and memory efficiency under Zipf-distributed cache workloads.

Experimental Settings

To evaluate the performance characteristics of Bloom Filters, Counting Bloom Filters, and Cuckoo Filters under realistic network-caching workloads, we designed an experimental setting that models how large-scale systems experience lookup operations, memory constraints, and traffic skew. The request stream is generated according to a Zipf distribution, which allows us to control the degree of popularity imbalance through a skew parameter α . Lower values of α produce nearly uniform access patterns, while higher values concentrate most queries on a small set of frequently accessed keys. This is important because modern CDNs, DNS resolvers, and web caches consistently observe heavy-tailed traffic, meaning the effectiveness of probabilistic filters can vary dramatically depending on whether the workload is uniform, moderately skewed, or extremely skewed.

The experiments use a fixed membership set of size n keys (the cache contents) and a disjoint non-member set (absent keys). Query workloads consist of $N = 50,000$ membership queries generated from a Zipf distribution over the member keys, with non-members interleaved to measure false positives in a realistic mixed stream. We evaluate two Zipf skew values, $\alpha \in \{1.0, 1.2\}$, to represent moderately skewed and more skewed request patterns.

To contextualize the potential performance gains of probabilistic pre-filters, we compare them against deterministically grounded baselines. The first baseline is a standard hash-table index representing the metadata layer of a conventional cache. The second baseline is a “no pre-filter” configuration in which every query directly accesses the hash table. For filter pipelines, the system performs: (1) filter membership check, and (2) hash-table lookup only if the filter returns positive. The key system-level metric is how many backend lookups the filter eliminates, and whether that reduction translates into lower end-to-end latency and higher throughput.

The internal configuration of each filter exposes adjustable design parameters. Bloom Filters vary bit-array size m and number of hash functions k . Counting Bloom Filters additionally vary counter size (we report 4-bit and 8-bit counter settings in Experiment 1). Cuckoo Filters vary fingerprint bits (8, 12, 16), with bucket size fixed at 4 and measured load factor reported. For fair comparison in Experiments 2 and 3, we select configurations that target approximately the same FPR (about 0.01). Specifically, we use Bloom configuration `bf_fp_0.01` (measured FPR 0.00945), Counting Bloom configuration `cbf_fp_0.01_c4` (measured FPR 0.00945), and Cuckoo configuration `cf_fpbits_12` (measured FPR 0.002175; closest available while staying below the 1% target).

For evaluation, we measure: (1) false-positive rate (FPR), (2) memory usage in bits per key using theoretical accounting (not `sys.getsizeof`), (3) average query latency (seconds per query) under Zipf workloads, (4) backend lookups per query (how often the hash table is consulted), and (5) throughput (queries/sec). To model backend miss costs in a stable and reproducible way, we use a deterministic busy-loop penalty of `miss_spins = 500` whenever the backend lookup is invoked and the key is absent.

Results

Experiment 1: Memory vs False Positive Rate

Figure 1 comes from a parameter sweep over each filter to see how memory (bits per key) trades off against the measured false positive rate (FPR). For Bloom filters, we can reach about 1% FPR using roughly 9.5852 bits per key. Counting Bloom filters can reach the same FPR, but they require much more memory because each position stores a counter instead of a single bit. Cuckoo filters reduce FPR quickly as fingerprint length increases; in particular, using 12-bit fingerprints produces a very low FPR (0.002175) at 13.3344 bits per key with load factor ≈ 0.90 , with no failed insertions in this run.

In Experiments 2 and 3, we compare filters under a similar operating point: Bloom and Counting Bloom are configured near 1% measured FPR (0.00945). For Cuckoo, the closest configuration we used that stays at or below the 1% target achieves an even lower FPR (0.002175).

Filter	Setting (used in Exp 2–3)	Bits/key	Measured FPR	Notes
Bloom	tuned to $\approx 1\%$ FPR	9.5852	0.00945	$k = 7$
Counting Bloom	tuned to $\approx 1\%$ FPR	38.3404	0.00945	4-bit counters
Cuckoo	12-bit fingerprints	13.3344	0.002175	bucket=4, $LF \approx 0.90$

Table 1: Configurations used for Experiments 2 and 3. Bloom and Counting Bloom are matched near 1% FPR. The Cuckoo configuration used is the closest setting in our sweep that stays at or below the 1% target.

Experiment 2: Lookup Latency Under Zipf Workloads

We measure average latency per query and backend lookups per query for $\alpha \in \{1.0, 1.2\}$ with $N = 50,000$ queries and a backend miss penalty modeled by `miss_spins` = 500. Figures 2 and 3 summarize the results.

For $\alpha = 1.0$, the average latency is $2.816 \mu\text{s}/\text{query}$ (Bloom), $2.201 \mu\text{s}/\text{query}$ (Counting Bloom), and $1.656 \mu\text{s}/\text{query}$ (Cuckoo). For $\alpha = 1.2$, the average latency is $2.841 \mu\text{s}/\text{query}$ (Bloom), $2.247 \mu\text{s}/\text{query}$ (Counting Bloom), and $1.628 \mu\text{s}/\text{query}$ (Cuckoo).

Backend lookups per query stay around 0.70 for Bloom and Counting Bloom, and are slightly lower for Cuckoo (about 0.68–0.69). This aligns with the lower FPR of the Cuckoo configuration in Table 1: fewer false positives means fewer unnecessary backend lookups, lowering end-to-end latency.

Experiment 3: End-to-End Throughput vs No-Filter Baseline

Experiment 3 compares throughput across four pipelines: (1) `no_filter` (hash table only), (2) Bloom + hash table, (3) Counting Bloom + hash table, and (4) Cuckoo + hash table. Figures 4 and 5 summarize throughput under each Zipf skew.

Under $\alpha = 1.0$, throughput increases from 273,371 qps (no filter) to 347,162 qps (Bloom+HT), 435,696 qps (Counting Bloom+HT), and 371,391 qps (Cuckoo+HT). Under $\alpha = 1.2$, throughput increases from 354,113 qps (no filter) to 355,889 qps (Bloom+HT), 451,349 qps (Counting Bloom+HT), and 623,109 qps (Cuckoo+HT).

The more skewed workload ($\alpha = 1.2$) produces the strongest system-level improvement for Cuckoo+HT. This supports the idea that heavy-tailed traffic makes pre-filtering more valuable, because small differences in backend lookup frequency and per-query overhead can compound into large throughput gains at scale.

Figures

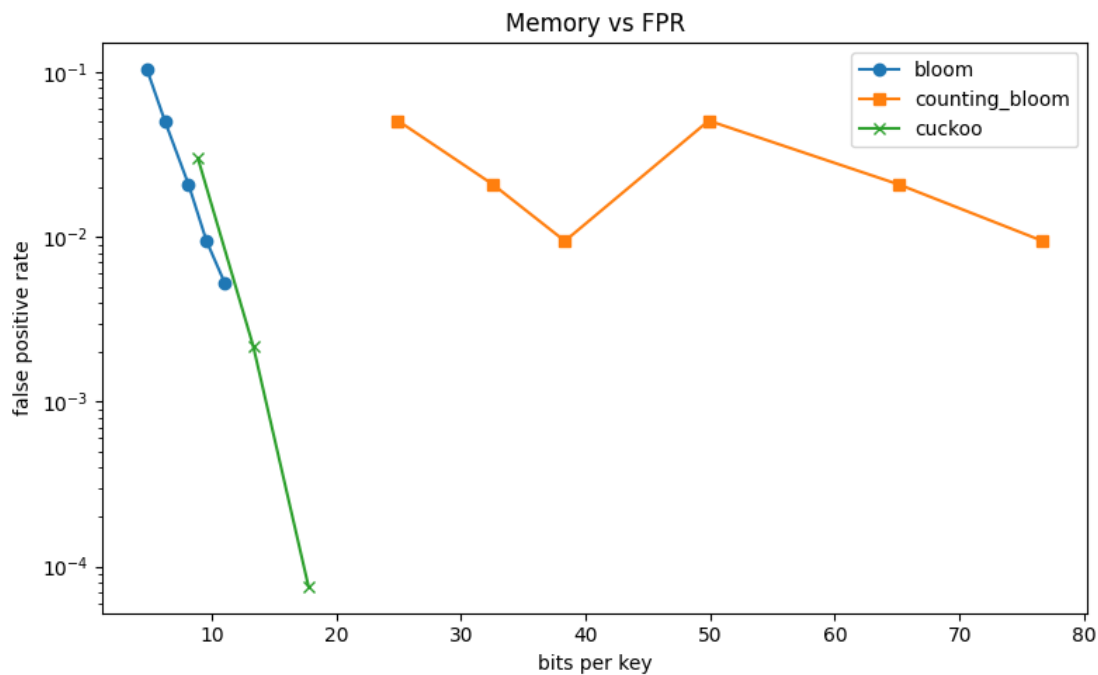


Figure 1: Experiment 1: Memory vs false positive rate for Bloom, Counting Bloom, and Cuckoo. This plot summarizes the parameter sweep and motivates the matched-FPR configuration selection used in later experiments.

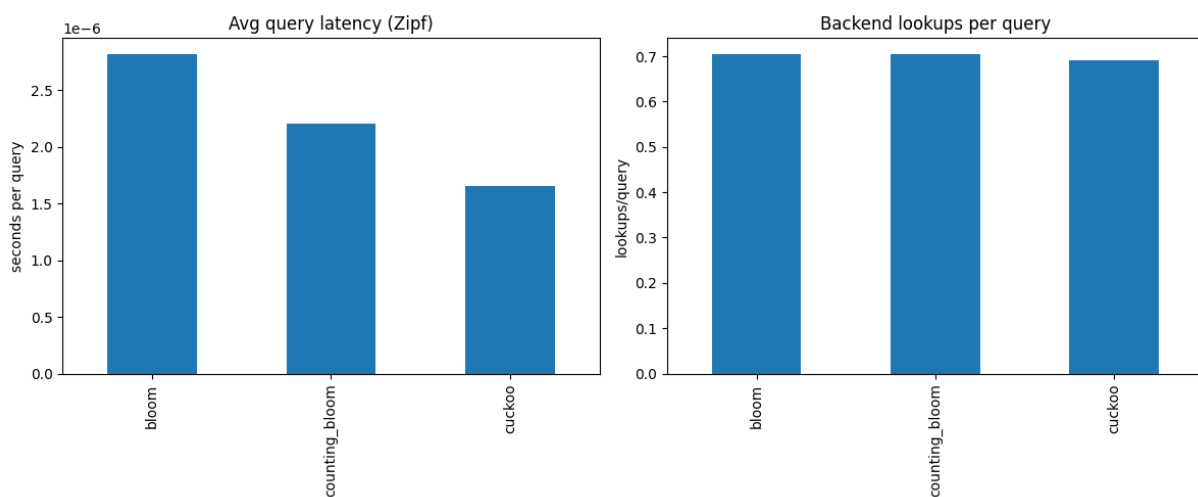


Figure 2: Experiment 2 ($\alpha = 1.0$, target FPR ≈ 0.01): average query latency and backend lookups per query for Bloom, Counting Bloom, and Cuckoo.

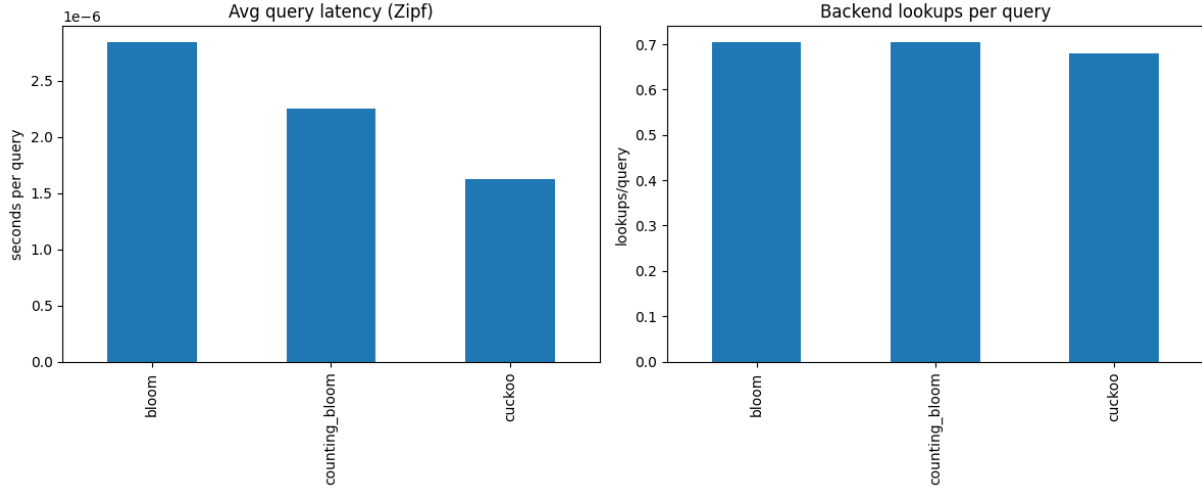


Figure 3: Experiment 2 ($\alpha = 1.2$, target FPR ≈ 0.01): average query latency and backend lookups per query for Bloom, Counting Bloom, and Cuckoo.

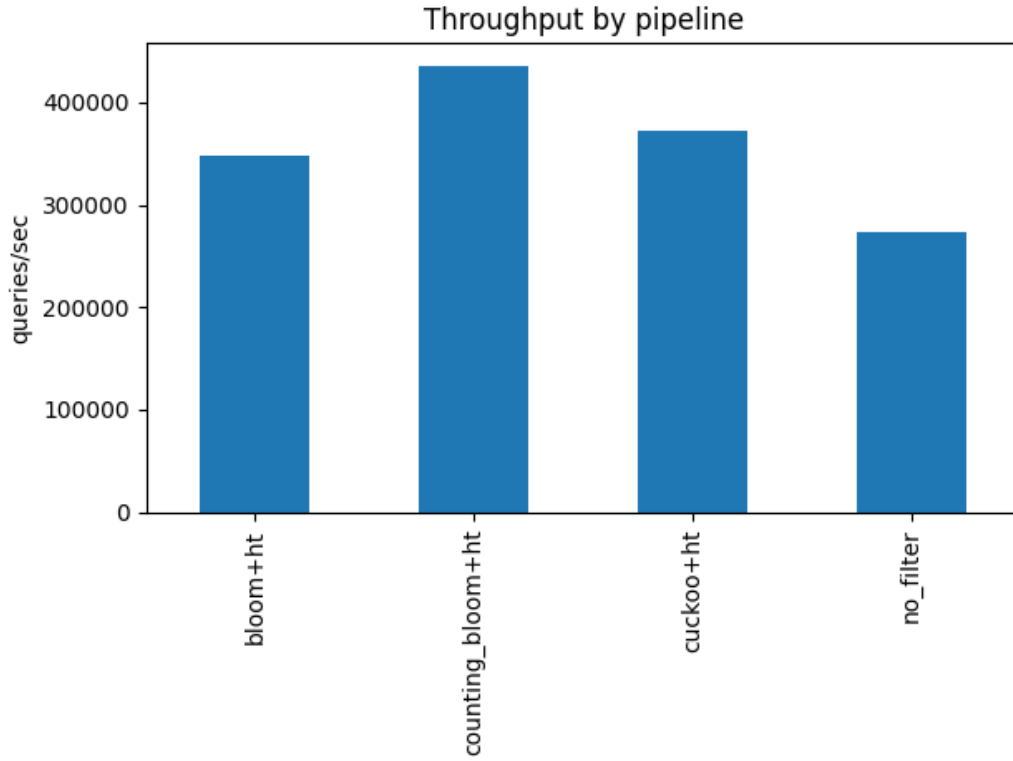


Figure 4: Experiment 3 ($\alpha = 1.0$): throughput (queries/sec) by pipeline, showing the end-to-end system effect of pre-filtering relative to no-filter.

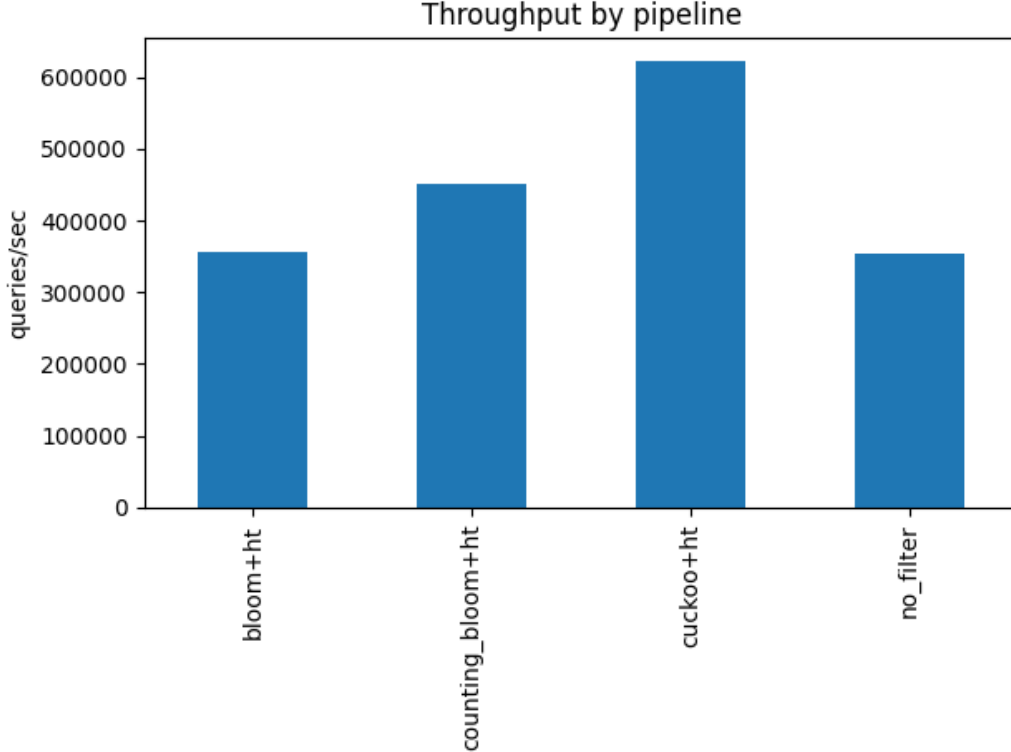


Figure 5: Experiment 3 ($\alpha = 1.2$): throughput (queries/sec) by pipeline, showing the strongest system-level improvement under higher Zipf skew.

Conclusion

Overall, the results support the hypothesis that Cuckoo Filters outperform Bloom and Counting Bloom Filters in lookup latency under Zipf-distributed cache workloads. In Experiment 2, Cuckoo achieves the lowest per-query latency for both $\alpha = 1.0$ and $\alpha = 1.2$, and it also reduces backend lookups per query relative to Bloom-based variants in our fixed configurations.

Experiment 1 shows the main memory trade-off clearly: Bloom reaches about 1% FPR with low memory (9.5852 bits/key), Counting Bloom matches the same FPR but requires much more memory because counters replace bits, and Cuckoo reaches very low FPR at moderate bits-per-key with a high load factor and no failed inserts in this setting. In Experiment 3, probabilistic pre-filters improve throughput relative to the no-filter baseline, and the effect depends on workload skew: Counting Bloom+HT is best at $\alpha = 1.0$, while Cuckoo+HT has the strongest improvement at $\alpha = 1.2$ and achieves the highest throughput there. So in our final conclusion, these experiments show that Zipf skew materially changes the system-level impact of filtering, and that Cuckoo filters are especially strong when the workload is more heavy-tailed.

References

- [1] F. Chang et al. Bigtable: A Distributed Storage System for Structured Data. *OSDI*, 2006.
- [2] G. DeCandia et al. Dynamo: Amazon’s Highly Available Key-Value Store. *SOSP*, 2007.

- [3] A. Lakshman and P. Malik. Cassandra: A Decentralized Structured Storage System. *SIGOPS*, 2010.
- [4] D. Karger et al. Consistent Hashing and Random Trees. *STOC*, 1997.
- [5] B. H. Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 1970.
- [6] A. Broder and M. Mitzenmacher. Network Applications of Bloom Filters: A Survey. *Internet Mathematics*, 2004.
- [7] L. Fan et al. Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol. *IEEE/ACM Transactions on Networking*, 2000.
- [8] B. Fan, D. Andersen, M. Kaminsky, and M. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. *CoNEXT*, 2014.